

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPD801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

29th May 2026



Abstract

Words words words words words words words words
Ord ord ord ord ord ord ord ord ord ord

CONTENTS

A	Code Availability	3
B	Lean File Structure	3
C	Common Lean Notation	3
D	Selected Core Definitions	6
D.1	Session Types	6
D.2	Processes	6
D.3	Judgements	7
D.4	Transition Labels	9
D.5	Transition Systems	10
E	Session Fidelity Theorems	15
F	Selected Proof Infrastructure	15

A Code Availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: <https://github.com/MikkelBrixNielsen/LeanPiLL>

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions are available in the repository history. These earlier versions are not part of the submitted artefact.

The appendix contains selected excerpts of the development. These excerpts are typeset to closely reflect the corresponding Lean code. In particular, some Unicode notation and spacing is rendered typographically rather than shown as directly executable Lean source. The complete directly executable Lean files, including all definitions, auxiliary lemmas, and proof scripts, are available in the accompanying codebase.

B Lean File Structure

The Lean development is organised into three main parts. The `Model/` directory contains the session types, processes, environments, hyperenvironments, and typing judgement. The `Semantics/` directory contains the transition labels and the process-, environment-, and judgement-level transition systems. The `SessionFidelity/` directory contains the main metatheoretic results relating these transition systems.

```
PiLL/
  Model/
    Environments/
    HyperEnvironments/
    Judgement/
    Processes/
    STypes/
    Base.lean
  Semantics/
    EnvStep/
    JudgementStep/
    ProcStep/
    Labels.lean
  SessionFidelity/
    ProcStep.lean
    EnvStep.lean
    Typability.lean
    SubjectReduction.lean
```

Listing 1. Top-level structure of the Lean development.

C Common Lean Notation

The notation is purely an overloading layer. Each syntactic category supplies its own instance, while the displayed symbols allow the same operation to be written uniformly throughout the formalisation.

Type Classes. The development uses a collection of typeclasses to overload recurring operations across syntactic categories. This allows shifting, substitution, opening, closing, prefix notation, and permutation to be written uniformly for types, processes, environments, hyperenvironments, actions, and labels. The following listing gives the common notation layer used throughout the later appendix excerpts.

```

148 class HasShift (Subject : Type) where
149   shift : Subject -> Nat -> Nat -> Subject
150
151 notation: max S: max " ↑ " d: max ", " c: max => HasShift.shift S d c
152 notation: max S: max " ↑ " c: max => HasShift.shift S 0 c
153 notation: max S: max "+" => HasShift.shift S 0 1
154
155 class HasSubst (Subject Replacement Target : Type) where
156   subst : Subject -> Replacement -> Target -> Subject
157
158 notation: max S "{" R " //" " T "}" => HasSubst.subst S R T
159
160 class HasOpen (Subject Replacement Target : Type) where
161   open_ : Subject -> Replacement -> Target -> Subject
162
163 notation: max S: max " (" T " | " R ")" => HasOpen.open_ S R T
164 notation: max S: max " (" R ")" => HasOpen.open_ S R 0
165
166 class HasClose (Subject Replacement Target : Type) where
167   close_ : Subject -> Replacement -> Target -> Subject
168
169 notation: max S: max " (« T " | " R »)" => HasClose.close_ S R T
170 notation: max S: max " (« R »)" => HasClose.close_ S R 0
171
172 class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
173   open_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
174
175 notation: max S: max " (" T " | " R1 ", " R2 ")" => HasOpenTwo.open_ S R1 R2 T
176 notation: max S: max " (" R1 ", " R2 ")" => HasOpenTwo.open_ S R1 R2 0
177
178 class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
179   close_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
180
181 notation: max S: max " (« T " | " R1 ", " R2 »)" => HasCloseTwo.close_ S R1 R2 T
182 notation: max S: max " (« R1 ", " R2 »)" => HasCloseTwo.close_ S R1 R2 0
183
184 class HasBracket (Subject Content Result : Type) where
185   brack : Subject -> Content -> Result
186
187 notation: 80 x "[" "]" => HasBracket.brack x ( )
188 notation: 80 x "[" "]" => HasBracket.brack x y
189 notation: 80 x "(" ")" => HasParen.paren x ( )
190 notation: 80 x "(" ")" => HasParen.paren x y
191
192 class HasPerm ( $\alpha$  : Type) where
193   perm :  $\alpha$  ->  $\alpha$  -> Prop
194
195 infixr: 54 " ~ " => HasPerm.perm
196

```

Listing 2. Common typeclasses and notation used throughout the Lean development. label

Type Notation. The notation in the following listing presents the Lean constructors for session types using the symbols adopted throughout the thesis. In particular, it defines notation for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication, allowing later definitions and theorem statements to closely resemble their mathematical presentation.

```
instance : One Types := ⟨Types.one⟩
instance : Bot Types := ⟨Types.bot⟩
infixr:90 " ⊗ " => Types.tensor
infixr:90 " ⊕ " => Types.oplus
infixr:90 " ⋈ " => Types.parr
infixr:90 " & " => Types.amp
prefix:95 "?? " => Types.quest
prefix:95 "!! " => Types.bang
prefix:max "∃." => Types.exists_
prefix:max "∀." => Types.forall_
postfix:max "⊥" => Types.dual
infix:90 " ⊸ " => Types.linImpl
```

Listing 3. Notation for session-type constructors. These declarations provide concrete syntax for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication.

Process Notation. The following listing defines the concrete Lean notation used for process terms. The notation mirrors the process syntax introduced in the theoretical presentation, including prefix forms, restriction, branching, server actions, links, parallel composition, and the inactive process. This notation is used throughout the later typing and transition-system listings.

```
notation:80 x "[[]]." P => Proc.one x P
notation:80 x "[[•]]." P => Proc.tensor x P
notation:80 x "[[ A ]]." P => Proc.output x P A
notation:80 x "[()]." P => Proc.bot x P
notation:80 x "[[•]]." P => Proc.parr x P
notation:80 x "[[◊]]." P => Proc.input x P

notation:75 "v([•,•]) " P:80 => Proc.cut P
notation:80 x "[[L]]." P:80 => Proc.selectL x P
notation:80 x "[[R]]." P:80 => Proc.selectR x P
notation:80 x "[[USE]]." P:80 => Proc.consume x P
notation:80 x "[[DUP]]([•])." P:80 => Proc.duplicate x P
notation:80 x "[[DISP]]." P:80 => Proc.dispose x P
notation:80 "!" x "{ P:80 }" => Proc.server x ∅ P
notation:80 "!" x "< zs >.{ P:80 }" => Proc.server x zs P
notation:80 x ".case{L " : " P:80 ", " "R" " : " Q :80}" => Proc.amp x P Q

notation:80 x "<↔p" y => Proc.link x y
infixr:70 " |p " => Proc.par
notation "0" => Proc.nil
```

Listing 4. Notation for process constructors. These declarations provide concrete syntax for prefixes, restriction, branching, server actions, links, parallel composition, and the inactive process.

D Selected Core Definitions

This appendix collects representative Lean definitions for the core syntactic objects used throughout the formalisation. The listings are intended to show the shape of the mechanised syntax and judgements.

D.1 Session Types

This listing gives the core Lean representation of session types. Type variables are represented using the locally nameless approach discussed in ??, separating free type variables from bound De Bruijn indices.

```

abbrev Atom := Nat
abbrev FTVar := Nat
abbrev BTVar := Nat

inductive TVar : Type
| free    (x : FTVar)
| bound   (x : BTVar)
deriving DecidableEq, Repr

inductive Types : Type where
| atom      (a : Atom)
| atomDual  (a : Atom)
| var       (v : TVar)
| varDual   (v : TVar)
| one
| bot
| tensor    (A B : Types)
| parr      (A B : Types)
| oplus     (A B : Types)
| amp       (A B : Types)
| bang      (A : Types)
| quest     (A : Types)
| forall_   (A : Types)
| exists_   (A : Types)
deriving DecidableEq, BEq, Repr

```

Listing 5. Locally nameless representation of type variables and the Lean definition of session types.

D.2 Processes

This listing gives the core Lean representation of channels and process syntax. Channels are represented using the locally nameless approach discussed in ??, separating free names from bound indices.

```

abbrev FPName := Nat
abbrev BPName := Nat

```

```

295
296 inductive Channel : Type where
297   | free    (x : FPName)
298   | bound   (x : BPName)
299   deriving DecidableEq, BEq, Repr
300
301 prefix:max "#" => Channel.free
302 prefix:max "$" => Channel.bound
303
304 inductive Proc : Type where
305   | nil
306   | one      (x : Channel) (P : Proc)
307   | bot      (x : Channel) (P : Proc)
308   | tensor   (x : Channel) (P : Proc)
309   | parr     (x : Channel) (P : Proc)
310   | cut      (P : Proc)
311   | par      (P Q : Proc)
312   | selectL  (x : Channel) (P : Proc)
313   | selectR  (x : Channel) (P : Proc)
314   | amp      (x : Channel) (P Q : Proc)
315   | output   (x : Channel) (P : Proc) (A : Types)
316   | input    (x : Channel) (P : Proc)
317   | server   (x : Channel) (zs : Finset Channel) (P : Proc)
318   | consume  (x : Channel) (P : Proc)
319   | duplicate (x : Channel) (P : Proc)
320   | dispose  (x : Channel) (P : Proc)
321   | link     (x y : Channel)
322   deriving DecidableEq, BEq

```

Listing 6. Locally nameless representation of channels and the Lean definition of process syntax.

D.3 Judgements

This listing gives the Lean definition of the main typing judgement, including the constructors for the static π LL typing relation and the exchange rules used to account for list-based environments.

```

328 inductive Typing : Nat -> Proc -> HyperEnv -> Prop where
329   | mix0 {n : Nat} :
330     Typing n 0 ∅
331
332   | mix {G H : HyperEnv} {P Q : Proc} {n : Nat} (hD : G.disjoint H) :
333     Typing n P G -> Typing n Q H ->
334     Typing n (P |p Q) (G |h H)
335
336   | one {P : Proc} {x : FPName} {n : Nat} :
337     Typing n P ∅ ->
338     Typing n (#x[[]].P) [[x:1]]
339
340   | bot {Γ : Env} {P : Proc} {x : FPName} {n : Nat} (hF : x ∉ Γ.names) :
341     Typing n P [Γ] ->
342     Typing n (#x[()].P) [x:⊥::Γ]
343

```



```

344 | cut {G : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat}
345   (L : Finset Fpname) :
346   (∀ x ∉ L, ∀ y ∉ L, x ≠ y →
347     Typing n (P((#x, #y))) (G |h [x:A::Γ] |h [y:A⊥::Δ])) →
348   Typing n (v((•,•)) P) (G |h [Γ,Δ])
349
350 | tensor {Γ Δ : Env} {P : Proc} {x : Fpname} {A B : Types} {n : Nat}
351   (hF : x ∉ Γ.names ∧ x ∉ Δ.names) (L : Finset Fpname) :
352   (∀ y ∉ L, Typing n (P((#y))) ([y:A::Γ] |h [x:B::Δ])) →
353   Typing n (#x[•]).P [x:A ⊗ B::Γ,Δ]
354
355 | parr {Γ : Env} {P : Proc} {x : Fpname} {A B : Types}
356   {n : Nat} (hF : x ∉ Γ.names) (L : Finset Fpname) :
357   (∀ y ∉ L, Typing n (P((#y))) [y:A::x:B::Γ]) →
358   Typing n (#x((•)).P) [x:A ∗ B::Γ]
359
360 | oplus1 {Γ : Env} {P : Proc} {x : Fpname} {A B : Types} {n : Nat} :
361   B.lc n → Typing n P [x:A::Γ] →
362   Typing n (#x[L]).P [x:A ⊕ B::Γ]
363
364 | oplus2 {Γ : Env} {P : Proc} {x : Fpname} {A B : Types} {n : Nat} :
365   A.lc n → Typing n P [x:B::Γ] →
366   Typing n (#x[R]).P [x:A ⊕ B::Γ]
367
368 | amp {Γ : Env} {P Q : Proc} {x : Fpname} {A B : Types} {n : Nat} :
369   Typing n P [x:A::Γ] → Typing n Q [x:B::Γ] →
370   Typing n (#x.case{L : P, R : Q}) [x:A & B::Γ]
371
372 | quest {Γ : Env} {P : Proc} {x : Fpname} {A : Types} {n : Nat} :
373   Typing n P [x:A::Γ] →
374   Typing n (#x[USE]).P [x:??A::Γ]
375
376 | bang {Γ : Env} {P : Proc} {x : Fpname} {A : Types} {n : Nat} :
377   ?eΓ → Typing n P [x:A::Γ] →
378   Typing n (!#x<Γ.names.image Channel.free>.{P}) [x:!!A::Γ]
379
380 | w {Γ : Env} {P : Proc} {x : Fpname} {A : Types} {n : Nat}
381   (hF : x ∉ Γ.names) :
382   A.lc n → Typing n P [Γ] →
383   Typing n (#x[DISP]).P [x:??A::Γ]
384
385 | c {Γ : Env} {P : Proc} {x : Fpname} {A : Types} {n : Nat}
386   (hF : x ∉ Γ.names) (L : Finset Fpname) :
387   (∀ y ∉ L, Typing n P((#y)) [x:??A::y:??A::Γ]) →
388   Typing n (#x[DUP](•).P) [x:??A::Γ]
389
390 | exists_ {Γ : Env} {P : Proc} {x : Fpname} {A B : Types} {n : Nat} :
391   A.lc n → Typing n P [x:B{A // 0}::Γ] →
392   Typing n (#x[A]).P [x:∃.B::Γ]
393
394 | forall_ {Γ : Env} {P : Proc} {x : Fpname} {B : Types} {n : Nat} :
395   Typing (n + 1) P [x:B::Γ+] →

```

```

393   Typing n (#x(( $\diamond$ )).P) [x:V.B:: $\Gamma$ ]
394
395 | ax {x y : FPNName} {A : Types} {n : Nat} (hneq : x  $\neq$  y) :
396   A.lc n ->
397   Typing n (#x  $\leftrightarrow_p$  #y) [x:A $^\perp$ :: [y:A]]
398
399 | exchange_env {G : HyperEnv} { $\Gamma$   $\Delta$  : Env} {P : Proc} {n : Nat} :
400   Typing n P ( $\Gamma :: G$ ) ->  $\Gamma \sim \Delta$  ->
401   Typing n P ( $\Delta :: G$ )
402
403 | exchange_hyper {G H : HyperEnv} {P : Proc} {n : Nat} :
404   Typing n P G -> G  $\sim$  H ->
405   Typing n P H
406
407 notation 50 n "  $\vdash$  " P " :: " G => Typing n P G

```

Listing 7. The main typing judgement. The judgement `Typing n P G` states that process `P` is well typed under hyperenvironment `G` at type-variable depth `n`.

```

411 def proc {G : HyperEnv} {P : Proc} {n : Nat}
412   (_ : n  $\vdash$  P :: G) : Proc := P
413
414 def env {G : HyperEnv} {P : Proc} {n : Nat}
415   (_ : n  $\vdash$  P :: G) : HyperEnv := G

```

Listing 8. Projection functions from typing derivations. Since the judgement is indexed by both the process and hyperenvironment, these projections simply recover the corresponding indices.

D.4 Transition Labels

This listing gives the core Lean definitions of the label and action datatypes discussed the paragraph on labels in ??, together with the associated free-name, introduced-name, and well-formedness functions used by the operational semantics.

```

425 inductive Mu : Type
426 | L
427 | R
428 | DISP
429 | DUP
430 | USE
431 deriving Repr, DecidableEq
432
433 inductive Act : Type
434 | one (x : FPNName)
435 | bot (x : FPNName)
436 | tensor (x y : FPNName)
437 | parr (x y : FPNName)
438 | output (x : FPNName) (A : Types)
439 | input (x : FPNName) (A : Types)
440 | muBrack (x : FPNName) ( $\mu$  : Mu)
441 | muParen (x : FPNName) ( $\mu$  : Mu)

```

```

442 deriving Repr, DecidableEq
443
444 inductive Lbl : Type
445 | tau
446 | link (x y : FPNName)
447 | act (p : Act)
448 | par (l l' : Act)
449 deriving Repr, DecidableEq
450
451 @[simp] def fNamesAct : Act -> Finset FPNName -- free names
452 | .one x | .bot x
453 | .tensor x _ | .parr x _
454 | .input x _ | .output x _
455 | .muBrack x _ | .muParen x _ => {x}
456
457 @[simp] def iNamesAct : Act -> Finset FPNName -- introduced names
458 | .one _ | .bot _ | .input _ _ | .output _ _
459 | Act.muBrack _ _ | Act.muParen _ _ => ∅
460 | .tensor _ y | .parr _ y => {y}
461
462 @[simp] def Lbl.f : Lbl -> Finset FPNName
463 | .tau => ∅
464 | .link x y => {x, y}
465 | .act a => fNamesAct a
466 | .par l l' => fNamesAct l ∪ fNamesAct l'
467
468 @[simp] def Lbl.i : Lbl -> Finset FPNName
469 | .tau => ∅
470 | .link _ _ => ∅
471 | .act a => iNamesAct a
472 | .par l l' => iNamesAct l ∪ iNamesAct l'
473
474 @[simp] def Lbl.WF : Lbl -> Prop
475 | .tau => True
476 | .link _ _ => True
477 | .act _ => True
478 | .par l l' => (iNamesAct l) ∩ (iNamesAct l') = ∅

```

Listing 9. Definitions of transition labels, actions, and their associated free and introduced names.

D.5 Transition Systems

This listing collects the full inductive definitions of the three transition relations for π MLL, as referenced from ??.

```

483 inductive TypingStepm :
484   {n : Nat} -> {G : HyperEnv} -> {P : Proc} -> Typing n P G -> Lbl ->
485   {n' : Nat} -> {G' : HyperEnv} -> {P' : Proc} -> Typing n' P' G' -> Prop where
486 | one {P : Proc} {x : FPNName} {n : Nat} {D : Typing n P ∅} :
487   TypingStepm (Typing.one (x := x) D) (x[]) D
488
489 | bot {Γ : Env} {P : Proc} {x : FPNName} {n : Nat}

```

```

491   {hF : x ∉ Γ.names} {D : Typing n P [Γ]} :
492   TypingStepm (Typing.bot (x := x) hF D) (x(⟦⟦)) D
493
494 | tensor {Γ Δ : Env} {P : Proc} {x y : FPNName} {A B : Types} {n : Nat}
495   {hF : x ∉ Γ.names ∧ x ∉ Δ.names} {L : Finset FPNName}
496   {huniq : ∀ z, z ∉ L → Typing n (P(⟦z⟦)) ([z:A::Γ] |h [x:B::Δ])}
497   {hy : y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names)} :
498   TypingStepm (Typing.tensor hF L huniq) (x(⟦y⟦)) (Typing_tensor_all_fresh huniq y hy
499   )
500
501 | parr {Γ : Env} {P : Proc} {x y : FPNName} {A B : Types} {n : Nat}
502   {hF : x ∉ Γ.names} {L : Finset FPNName}
503   {huniq : ∀ z, z ∉ L → Typing n (P(⟦z⟦)) [z : A::x : B::Γ]}
504   {hy : y ∉ ({x} ∪ P.f ∪ Γ.names)} :
505   TypingStepm (Typing.parr hF L huniq) (x(⟦y⟦)) (Typing_parr_all_fresh huniq y hy)
506
507 | par1 {G H G' : HyperEnv} {P Q P' : Proc} {l : Lbl} {n : Nat}
508   {hD1 : G.disjoint H} {hD2 : G'.disjoint H}
509   {D : Typing n P G} {D' : Typing n P' G'} {E : Typing n Q H}
510   (h : TypingStepm D l D') (disj : l.i ∩ Q.f = ∅) :
511   TypingStepm (Typing.mix hD1 D E) l (Typing.mix hD2 D' E)
512
513 | par2 {G H H' : HyperEnv} {P Q Q' : Proc} {l : Lbl} {n : Nat}
514   {hD1 : G.disjoint H} {hD2 : G'.disjoint H'}
515   {D : Typing n P G} {E : Typing n Q H} {E' : Typing n Q' H'}
516   (h : TypingStepm E l E') (disj : l.i ∩ P.f = ∅) :
517   TypingStepm (Typing.mix hD1 D E) l (Typing.mix hD2 D E')
518
519 | syn {G G' H H' : HyperEnv} {P P' Q Q' : Proc} {l l' : Act} {n : Nat}
520   {hD1 : G.disjoint H} {hD2 : G'.disjoint H'}
521   {D : Typing n P G} {D' : Typing n P' G'}
522   {E : Typing n Q H} {E' : Typing n Q' H'}
523   (h1 : TypingStepm D l D') (h2 : TypingStepm E l' E')
524   (disj : (l |1 l').i ∩ (P |p Q).f = ∅) (WF : (l |1 l').WF) :
525   TypingStepm (Typing.mix hD1 D E) (l |1 l') (Typing.mix hD2 D' E')
526
527 | one_bot {G : HyperEnv} {Γ : Env} {P P' : Proc} {n : Nat} {L : Finset FPNName}
528   {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
529     Typing n (P(⟦x, #y⟦)) (G |h [x:1::∅] |h [y:⊥::Γ])}
530   {x y : FPNName}
531   {hx : x ∉ ({y} ∪ P.f ∪ G.names ∪ Γ.names)}
532   {hy : y ∉ (P.f ∪ G.names ∪ Γ.names)}
533   {D' : Typing n P' (G |h [∅, Γ])}
534   (hStep : TypingStepm (Typing_one_bot_all_fresh huniq x y hx hy)
535     (x(⟦⟦ |1 y(⟦⟦) D')) :
536   TypingStepm (Typing.cut L huniq) (τ) D'
537
538 | tensor_parr {G : HyperEnv} {Γ Δ Ξ : Env} {P P' : Proc} {A B : Types}
539   {n : Nat} {L : Finset FPNName}
540   {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
541     Typing n (P(⟦x, #y⟦))
542     (G |h [x:A⊗B::Γ,Δ] |h [y:A⊥⊗B⊥::Ξ])}

```

```

540 {huniq' : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
541       ∀ x' ∉ L, ∀ y' ∉ L, x' ≠ y' →
542       x ≠ x' → x ≠ y' → y ≠ x' → y ≠ y' →
543       Typing n (P'((2 | #x, #y)((#x', #y'))))
544       (G |h [x:B::Δ] |h [x':A::Γ] |h [y':A⊥::y:B⊥::Ξ]))}
545 {x y : FPNName} (hx : x ∉ L) (hy : y ∉ L) (hneq : x ≠ y)
546 {x' y' : FPNName} (hx' : x' ∉ L) (hy' : y' ∉ L) (hneq' : x' ≠ y')
547 (hxx' : x ≠ x') (hxy' : x ≠ y') (hyx' : y ≠ x') (hyy' : y ≠ y')
548 (hxP : x ∉ P.f) (hyP : y ∉ P.f) (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
549 (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f) (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
550 (hStep : TypingStepm (huniq x hx y hy hneq)
551   (x[[x']] |1 y((y'))))
552 (huniq' x hx y hy hneq x' hx' y' hy' hneq' hxx' hxy' hyx' hyy')) :
553 TypingStepm (Typing.cut L huniq) (τ)
554 (Typing_double_cut_tensor_parr huniq')
555 | res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env} {P P' : Proc} {A : Types}
556 {n : Nat} {l : Lbl} {L L' : Finset FPNName}
557 {huniq : ∀ z ∉ L, ∀ w ∉ L, z ≠ w →
558   Typing n (P((#z, #w)) (G |h [z:A::Γ] |h [w:A⊥::Δ]))}
559 {huniq' : ∀ z ∉ L', ∀ w ∉ L', z ≠ w →
560   Typing n (P'((#z, #w)) (G' |h [z:A::Γ'] |h [w:A⊥::Δ']))}
561 {x y : FPNName} (hneq : x ≠ y)
562 (hx_pre : x ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
563 (hy_pre : y ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
564 (hx_post : x ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
565 (hy_post : y ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
566 (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i)
567 (hStep : TypingStepm
568   (Typing_res_all_fresh huniq x y hx_pre hy_pre hneq) l
569   (Typing_res_all_fresh huniq' x y hx_post hy_post hneq)) :
570 TypingStepm (Typing.cut L huniq) l (Typing.cut L' huniq')
571 | perm_env {G H : HyperEnv} {Γ Γ' : Env} {P P' : Proc} {n n' : Nat} {l : Lbl}
572 {D : n ⊢ P :: Γ :: G} {D' : n' ⊢ P' :: H} (hP1 : Γ ~ Γ')
573 (hTS : TypingStepm D l D') :
574 TypingStepm (Typing.exchange_env D hP1) l D'
575 | perm_hyper {G G' H H' : HyperEnv} {P P' : Proc} {n n' : Nat} {l : Lbl}
576 {D : n ⊢ P ⊢ G} {D' : n' ⊢ P' ⊢ H}
577 (hP1 : G ~ G') (hP2 : H ~ H')
578 (hTS : TypingStepm D l D') :
579 TypingStepm (Typing.exchange_hyper D hP1) l
580 (Typing.exchange_hyper D' hP2)

```

Listing 10. The typed transition relation TypingStep_m .

```

581 inductive EnvStepm : HyperEnv → Lbl → HyperEnv → Prop where
582 | one {x : FPNName} :
583   EnvStepm [[x:1]] (x[[ ]]) ∅
584 | bot {Γ : Env} {x : FPNName} :

```

```

589   EnvStepm [x:⊥::Γ] (x(⟦⟦)) [Γ]
590
591 | tensor {Γ Δ : Env} {x x' : FName} {A B : Types}
592   (hF : x' ∉ HyperEnv.names [x:A⊗B::Γ,Δ]) :
593   EnvStepm [x:A⊗B::Γ,Δ] (x[⟦x'⟦]) ([x':A::Γ] |h [x:B::Δ])
594
595 | parr {Γ : Env} {x x' : FName} {A B : Types}
596   (hF : x' ∉ HyperEnv.names [x:A⋈B::Γ]) :
597   EnvStepm [x:A⋈B::Γ] (x(⟦x'⟦)) [x':A::x:B::Γ]
598
599 | par1 {G G' H : HyperEnv} {l : Lbl} :
600   EnvStepm G l G' ->
601   EnvStepm (G |h H) l (G' |h H)
602
603 | par2 {G H H' : HyperEnv} {l : Lbl} :
604   EnvStepm H l H' ->
605   EnvStepm (G |h H) l (G |h H')
606
607 | syn {G G' H H' : HyperEnv} {l l' : Act} :
608   EnvStepm G l G' -> EnvStepm H l' H' ->
609   EnvStepm (G |h H) (l |l l') (G' |h H')
610
611 | one_bot {G : HyperEnv} {Γ : Env} {x y : FName} :
612   EnvStepm (G |h [x:1]) |h [y:⊥::Γ] (x[⟦⟦] |1y(⟦⟦)) (G |h [Γ]) ->
613   EnvStepm (G |h [Γ]) (τ) (G |h [Γ])
614
615 | tensor_parr {G : HyperEnv} {Γ Δ Ξ : Env}
616   {x x' y y' : FName} {A B : Types} :
617   EnvStepm
618   (G |h [x:A⊗B::Γ,Δ] |h [y:A⊥⋈B⊥::Ξ])
619   (x[⟦x'⟦] |1y(⟦y'⟦))
620   (G |h [x:B::Δ] |h [x':A::Γ] |h [y':A⊥::y:B⊥::Ξ]) ->
621   EnvStepm (G |h [Γ,Δ,Ξ]) (τ) (G |h [Γ,Δ,Ξ])
622
623 | res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env} {x y : FName} {A : Types}
624   {l : Lbl} (hFx : x ∉ l.i ∪ l.f) (hFy : y ∉ l.i ∪ l.f) :
625   EnvStepm
626   (G |h [x:A::Γ] |h [y:A⊥::Δ])
627   1
628   (G' |h [x:A::Γ'] |h [y:A⊥::Δ']) ->
629   EnvStepm (G |h [Γ,Δ]) l (G' |h [Γ',Δ'])
630
631 | perm_env {G H : HyperEnv} {Γ Γ' : Env} {l : Lbl} (hP : Γ ~ Γ') :
632   EnvStepm (Γ::G) l H ->
633   EnvStepm (Γ'::G) l H
634
635 | perm_hyper {G G' H H' : HyperEnv} {l : Lbl} (hP1 : G ~ G') (hP2 : H ~ H') :
636   EnvStepm G l H ->
637   EnvStepm G' l H'

```

Listing 11. The environment-level transition relation EnvStep_m .

```

638 inductive ProcStepm : (P : Proc) -> Lbl -> (P' : Proc) -> Prop where
639 | one {P : Proc} {x : FPNName} :
640   ProcStepm (#x[[]].P) (x[[]]) P
641
642 | bot {P : Proc} {x : FPNName} :
643   ProcStepm (#x[() ].P) (x[() ]) P
644
645 | tensor {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
646   ProcStepm (#x[[•]].P) (x[[y]]) (P((#y)))
647
648 | parr {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
649   ProcStepm (#x[(•)].P) (x[(y)]) (P((#y)))
650
651 | par1 {P P' Q : Proc} {l : Lbl} :
652   ProcStepm P l P' -> l.i ∩ Q.f = ∅ ->
653   ProcStepm (P |p Q) l (P' |p Q)
654
655 | par2 {P Q Q' : Proc} {l : Lbl} :
656   ProcStepm Q l Q' -> l.i ∩ P.f = ∅ ->
657   ProcStepm (P |p Q) l (P |p Q')
658
659 | syn {P P' Q Q' : Proc} {l l' : Act} :
660   ProcStepm P l P' -> ProcStepm Q l' Q' ->
661   (l |1 l').i ∩ (P |p Q).f = ∅ -> (l |1 l').WF ->
662   ProcStepm (P |p Q) (l |1 l') (P' |p Q')
663
664 | one_bot {P P' : Proc} {x y : FPNName}
665   (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
666   ProcStepm (P((#x, #y))) (x[[]] |1 y[() ]) P' ->
667   ProcStepm (ν((•, •)) P) (τ) P'
668
669 | tensor_parr {P P' : Proc} {x x' y y' : FPNName}
670   (hxP : x ∉ P.f) (hyP : y ∉ P.f)
671   (hx'P : x' ∉ P'.f) (hy'P : y' ∉ P'.f)
672   (hxP' : x ∉ P'.f) (hyP' : y ∉ P.f)
673   (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
674   (hxx' : x ≠ x') (hxy : x ≠ y)
675   (hxy' : x ≠ y') (hxx' : x ≠ x')
676   (hyy' : y ≠ y') (hxy' : y ≠ x') :
677   ProcStepm (P((#x, #y))) (x[[x']] |1 y[(y')]) (P'((2 | #x, #y))((#x', #y'))) ->
678   ProcStepm (ν((•, •)) P) (τ) (ν((•, •)) (ν((•, •)) P'))
679
680 | res {P P' : Proc} {l : Lbl} {x y : FPNName} (hneq : x ≠ y)
681   (hx : x ∉ P.f ∪ P'.f) (hy : y ∉ P.f ∪ P'.f)
682   (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i) :
683   ProcStepm (P((#x, #y))) l (P'((#x, #y))) ->
684   ProcStepm (ν((•, •)) P) l (ν((•, •)) P')

```

Listing 12. The process-level transition relation ProcStep_m.

E Session Fidelity Theorems

The following listings give the Lean statements of the four main theorems establishing session fidelity for π MLL, as discussed in ?? . The erasure functions `proc` and `env` (Listing 8) extract the process and hyperenvironment from a typing derivation respectively.

The first two theorems show that typed transitions project to process-level and environment-level transitions. The third theorem is ment to prove typability subject reduction i.e. if a well-typed process steps it can be lifted to a typed transition. The final theorem packages these results into the usual subject-reduction statement.

```

theorem session_fidelity_proc_m
  {n n' : Nat} {G G' : HyperEnv} {P P' : Proc} {l : Lbl}
  {D : Typing n P G} {D' : Typing n' P' G'} (hStep : TypingStep_m D l D') :
  ProcStep_m (proc D) l (proc D') := by ...

```

Listing 13. Process erasure simulation: typed derivation steps project to process steps.

```

theorem session_fidelity_env_m
  {n n' : Nat} {G G' : HyperEnv} {P P' : Proc} {l : Lbl}
  {D : Typing n P G} {D' : Typing n' P' G'} (hStep : TypingStep_m D l D') :
  EnvStep_m (env D) l (env D') := by ...

```

Listing 14. Environment erasure simulation: typed derivation steps project to environment steps.

```

theorem typability_subject_reduction_m
  {n : Nat} {G : HyperEnv} {P P' : Proc} {l : Lbl}
  (D : Typing n P G) (hPS : ProcStep_m P l P') :
  ∃ (G' : HyperEnv) (D' : Typing n P' G'),
  TypingStep_m D l D' := by ...

```

Listing 15. Typability and subject reduction: process steps from well-typed processes lift to typed derivation steps.

```

theorem subject_reduction_m {n : Nat} {G : HyperEnv} {P P' : Proc} {l : Lbl}
  (D : Typing n P G) (hPS : ProcStep_m P l P') :
  ∃ G', EnvStep_m G l G' ∧ (Typing n P' G') := by
  obtain ⟨G', D', hTS⟩ := typability_subject_reduction_m D hPS
  have hES := session_fidelity_env_m hTS
  exact ⟨G', hES, D'⟩

```

Listing 16. Subject reduction for the multiplicative fragment. The result follows from Listings 14 and 15. label

F Selected Proof Infrastructure

This appendix collects representative lemma statements from the proof infrastructure supporting the session fidelity development, as discussed in ?? and ??.

Freshness bridge lemmas. The following lemmas bridge the mismatch between the co-finite quantification used in the typing rules and the concrete fresh names chosen by ProcStep_m . They show that a co-finite typing premise can be instantiated at any concrete name that is fresh for the process and the surrounding environments. These lemmas are used by the typed transition rules for prefix actions, where the process transition records a particular name, while the typing rule provides a premise for all names outside a finite exclusion set.

```

lemma Typing_tensor_all_fresh {n : Nat} {P : Proc} {Γ Δ : Env}
  {x : FPNName} {A B : Types} {L : Finset FPNName}
  (hT : ∀ z ∉ L, Typing n (P((#z))) ([z:A::Γ] |h [x:B::Δ])) :
  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names) ->
  Typing n (P((#y))) ([y:A::Γ] |h [x:B::Δ]) := by ...

lemma Typing_parr_all_fresh {n : Nat} {P : Proc} {Γ : Env}
  {x : FPNName} {A B : Types} {L : Finset FPNName}
  (hT : ∀ z ∉ L, Typing n (P((#z))) ([z:A::x:B::Γ])) :
  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names) ->
  Typing n (P((#y))) ([y:A::x:B::Γ]) := by ...

```

Listing 17. Freshness bridge lemmas for instantiating the co-finite premises of the tensor and parr typing rules.

All the *all-fresh* lemmas follow the same proof structure. They first choose the name or names required by the relevant constructor fresh from the co-finite exclusion set, the concrete target names, the free names of the process, and the names already present in the typing environment. The co-finite typing premise is then instantiated with these fresh names, and the resulting derivation is renamed to use the desired concrete names by applying substitution preservation for typing. The remaining proof obligations show that the substitutions have no effect on the surrounding typing environment or on the process body, except at the occurrences introduced by opening the process body. These obligations are discharged using the freshness assumptions together with the substitution and opening lemmas for processes and environments.

Typing inversion. Inversion lemmas recover the typing rule used to derive a given judgement. They are used throughout the typability proof to identify the typing rule compatible with the observed process shape.

The proofs proceed by induction on the typing derivation after first generalising the concrete process shape. All constructors whose conclusion cannot have the required process form are discharged by contradiction. The matching constructor case then uses injectivity of the process and channel constructors to recover the equalities between the displayed process and the process appearing in the typing derivation. For Typing_inv_one , this directly yields the empty continuation typing premise and a permutation showing that the hyperenvironment contains exactly the singleton environment for x . For Typing_inv_tensor , the matching case additionally exposes the hidden types, component environments, and co-finite premise of the original tensor rule. The exchange cases propagate the inversion result through environment and hyperenvironment permutation, composing the recovered permutation with the exchange permutation introduced by the typing derivation.

```

lemma Typing_inv_one {n : Nat} {P : Proc} {x : FPNName} {G : HyperEnv}
  (hT : Typing n (#x[]).P) G :
  (G ~ [[x:1]]) ∧ Typing n P ∅ := by ...

lemma Typing_inv_tensor {n : Nat} {P : Proc} {G : HyperEnv} {x : FPNName}

```

```

(hT : Typing n (#x[•]).P)  $\mathcal{G}$  :
 $\exists$  (A B : Types) ( $\Gamma \Delta$  : Env) (L : Finset FPNName),
( $\mathcal{G} \sim [x:A \otimes B::\Gamma, \Delta]$ )  $\wedge$  ( $\forall z \notin L$ , Typing n (P(#z)) ( $[z:A::\Gamma] \mid_h [x:B::\Delta]$ )) := by ...

```

Listing 18. Representative inversion lemmas for the typing relation. The one case recovers a singleton hyperenvironment and continuation typing derivation, while the tensor case recovers the hidden types, environments, and co-finite premise.

Hyperenvironment extraction. Extraction lemmas isolate specific components from a hyperenvironment permutation, exposing the typing context required for a particular transition case. The following statement is representative of the extraction lemmas used in the `tensor_parr` case of `typability_subject_reductionm`. Since hyperenvironments are represented as lists modulo permutation, the extracted components can appear in different relative positions. The lemma therefore returns a disjunction covering the possible arrangements.

The proof proceeds by membership extraction through the pre-transition permutation, followed by case analysis on the relative positions of the restricted endpoints. Freshness and inequality assumptions rule out impossible coincidences, while the valid branches reconstruct the resulting hyperenvironment permutations using permutation rotation, merge, and cancellation lemmas.

```

lemma HyperEnv.Perm.extract_tensor_parr_res
{ $\mathcal{G} \mathcal{H} \mathcal{G}_r$  : HyperEnv} { $\Gamma \Gamma' \Gamma'' \Delta \Delta' \Delta'' \Xi$  : Env}
{u v x x' y y' : FPNName} {A B C D E : Types}
(h_pre :  $\mathcal{G} \mid_h [u:E::\Gamma'] \mid_h [v:E^\perp::\Delta'] \sim$ 
 $\mathcal{G}_r \mid_h [x:A \otimes B::\Gamma, \Delta] \mid_h [y:C \wp D^\perp::\Xi]$ )
(h_post :  $\mathcal{H} \mid_h [u:E::\Gamma''] \mid_h [v:E^\perp::\Delta''] \sim$ 
 $\mathcal{G}_r \mid_h [x:B::\Delta] \mid_h [x':A::\Gamma] \mid_h [y':C::y:D::\Xi]$ )
(hux : u  $\neq$  x) (hux' : u  $\neq$  x') (huy : u  $\neq$  y) (huy' : u  $\neq$  y')
(hvx : v  $\neq$  x) (hvx' : v  $\neq$  x') (hvy : v  $\neq$  y) (hvy' : v  $\neq$  y')
(huv : u  $\neq$  v) (hFu : u  $\notin \mathcal{G}.names$ ) (hFv : v  $\notin \mathcal{G}.names$ )
(hFu' : u  $\notin \mathcal{H}.names$ ) (hFv' : v  $\notin \mathcal{H}.names$ )
(hu $\Delta'$  : u  $\notin \Delta'.names$ ) (hv $\Gamma'$  : v  $\notin \Gamma'.names$ ) :
( $\exists \mathcal{G}_n \Gamma_n \Delta_n \Xi_n$ ,
 $\mathcal{G} \mid_h [\Gamma', \Delta'] \sim \mathcal{G}_n \mid_h [x:A \otimes B::\Gamma_n, \Delta_n] \mid_h [y:C \wp D^\perp::\Xi_n] \wedge$ 
 $\mathcal{H} \mid_h [\Gamma'', \Delta''] \sim \mathcal{G}_n \mid_h [x:B::\Delta_n] \mid_h [x':A::\Gamma_n] \mid_h [y':C::y:D::\Xi_n]$ )
 $\vee$ 
( $\exists \mathcal{G}_n \Gamma_n \Delta_n \Xi_n$ ,
 $\mathcal{G} \mid_h [\Gamma', \Delta'] \sim \mathcal{G}_n \mid_h [x:A \otimes B::\Gamma_n, \Delta_n ++ y:C \wp D^\perp::\Xi_n] \wedge$ 
( $\mathcal{H} \mid_h [\Gamma'', \Delta''] \sim \mathcal{G}_n \mid_h [x:B::\Delta_n] \mid_h [x':A::\Gamma_n ++ y':C::y:D::\Xi_n] \vee$ 
 $\mathcal{H} \mid_h [\Gamma'', \Delta''] \sim \mathcal{G}_n \mid_h [x':A::\Gamma_n] \mid_h [x:B::\Delta_n ++ y':C::y:D::\Xi_n]$ )) := by ...

```

Listing 19. Representative hyperenvironment extraction lemma for the tensor-parr synchronisation case.